EAD

EDUCAÇÃO A DISTÂNCIA

UNIDADE IV

Programação Orientada a Objetos I

Prof. MSc. Tarcísio Peres

Conteúdo da Unidade IV

- Expressões lambda e LINQ.
- *Disposal*, *Garbage Collection*, *Debugging* e *Tracing*.
- *Patterns*, *Threading* e *Tasks*.
- Funções e padrões assíncronos.
- Classes e algoritmos.

Expressões lambda

- Uma expressão lambda é, em sua essência, uma forma compacta de definir um método.
- Ela separa os parâmetros da função de seu corpo (operador “=>”). Por exemplo, a expressão lambda (x, y) => x + y define uma função que pega dois parâmetros e retorna sua soma.
- Ao usar essa expressão, programadores podem criar funções sem precisar nomeá-las, o que é especialmente útil em operações curtas, usadas como argumento para métodos de ordem superior.
- Um dos principais lugares em que as expressões lambda são usadas é com *Language Integrated Query* (LINQ), uma série de extensões de métodos que permitem manipular coleções de dados de maneira declarativa.
    - Além da sintaxe concisa, o que realmente torna as expressões lambda poderosas é o fato de elas capturarem o contexto em que são definidas, ou seja, podem acessar variáveis do seu escopo circundante, permitindo a criação de *closures*. Essa capacidade de capturar o estado as torna extremamente flexíveis, permitindo aos programadores criar funções que se comportam de acordo com o contexto em que foram criadas.

LINQ: consultas e operadores

- LINQ é uma característica robusta do C# e parte fundamental do .NET Framework, permitindo consultas de dados de forma mais intuitiva e segura, de modo integrado à linguagem.
- Essa tecnologia é usada principalmente para buscar e filtrar dados de diferentes fontes, como coleções de objetos, bancos de dados e *Extensible Markup Language* (XML), de maneira declarativa e consistente.
- Também possibilita consultas SQL-*like* diretamente em coleções de objetos em C#, proporcionando uma sintaxe de consulta coerente e uniforme, apresentando uma série de operadores de consulta que permitem ações como seleção, filtragem, ordenação, agrupamento, entre outras, em diferentes fontes de dados.
  - Oferece suporte a consultas compostas e projetadas, viabilizando a criação de consultas complexas e versáteis.
  - Proporciona uma verificação de tipo em tempo de compilação, tornando o código mais seguro ao reduzir erros em tempo de execução, aumentando a qualidade e a manutenibilidade do código-fonte.

LINQ: consultas e operadores

Essa tecnologia tem diferentes provedores, como:

- LINQ to Objects (para trabalhar com coleções em memória).
- LINQ to SQL (para trabalhar com bancos de dados relacionais).
- LINQ to XML (para manipular dados em formato XML).

- Cada provedor tem suas particularidades, mas todos compartilham do mesmo princípio básico de prover uma interface de consulta integrada e uniforme.

- A manipulação e a consulta de dados tornam-se mais intuitivas, limpas e coesas, reduzindo a complexidade associada a operações de dados e aumentando a produtividade do desenvolvedor.

LINQ: consultas e operadores

```
37. class Deus
38. {
39.    public string Nome { get; set; }
40.    public string Dominio { get; set; }
41.    public string Simbolo { get; set; }
42. }
```

Fonte: autoria própria.

```
17.      // Uso do LINQ para encontrar todos os deuses cujo domínio é o céu
18.      var deusesDoCeu = from deus in deuses
19.                        where deus.Dominio == "Céu"
20.                        select deus;
21.      Console.WriteLine("Deuses do Céu:");
22.      foreach (var deus in deusesDoCeu)
23.      {
24.         Console.WriteLine(deus.Nome);
25.      }
26.      // Uso do LINQ para ordenar os deuses pelo nome
27.      var deusesOrdenados = from deus in deuses
28.                            orderby deus.Nome ascending
29.                            select deus;
30.      Console.WriteLine("\nDeuses Ordenados por Nome:");
31.      foreach (var deus in deusesOrdenados)
32.      {
33.         Console.WriteLine(deus.Nome);
34.      }
35.   }
36. }
```

LINQ: consultas e operadores

```
1.  using System;
2.  using System.Collections.Generic;
3.  using System.Linq;
4.  class Programa
5.  {
6.     static void Main()
7.     {
8.        // Criação de uma lista de deuses da mitologia grega
9.        List<Deus> deuses = new List<Deus>
10.       {
11.          new Deus { Nome = "Zeus", Dominio = "Céu", Simbolo = "Raio" },
12.          new Deus { Nome = "Poseidon", Dominio = "Mar", Simbolo =
"Tridente" },
13.          new Deus { Nome = "Hades", Dominio = "Submundo", Simbolo =
"Capacete da Invisibilidade" },
14.          new Deus { Nome = "Atena", Dominio = "Sabedoria", Simbolo =
"Coruja" },
15.          new Deus { Nome = "Apolo", Dominio = "Sol", Simbolo = "Lira" }
16.       };
```

Fonte: autoria própria.

LINQ: consultas e operadores

Operação	Descrição
Select	Projeção de dados. Permite selecionar uma ou mais propriedades de objetos em uma coleção
Where	Filtra a coleção com base em uma condição especificada
OrderBy / OrderByDescending	Ordena a coleção com base em uma chave especificada, em ordem crescente ou decrescente
GroupBy	Agrupa elementos da coleção pela chave especificada, criando grupos de elementos relacionados
Join	Combina elementos de duas coleções com base em uma chave comum, gerando um resultado com as combinações desejadas

Fonte: autoria própria.

LINQ: consultas e operadores

Operação	Descrição
Count	Conta o número de elementos na coleção que atendem a uma condição específica
Sum / Average / Min / Max	Realiza operações de agregação na coleção, como soma, média, mínimo e máximo de valores numéricos
Any	Verifica se algum elemento da coleção satisfaz uma condição especificada
All	Verifica se todos os elementos da coleção satisfazem uma condição especificada
Distinct	Retorna elementos distintos da coleção, removendo duplicatas
Take / Skip	Retorna uma quantidade específica de elementos da coleção ou pula uma quantidade especificada de elementos
First / FirstOrDefault	Retorna o primeiro elemento da coleção que atende a uma condição especificada. Se nenhum elemento for encontrado, pode retornar um valor-padrão
Single / SingleOrDefault	Retorna um único elemento da coleção que atende a uma condição especificada. Se houver mais de um ou nenhum elemento que satisfaça a condição, pode lançar uma exceção
ToList / ToArray	Converte a coleção em uma lista ou array, permitindo maior flexibilidade para trabalhar com os dados

Fonte: autoria própria.

Garbage Collection

- Na plataforma .NET – da qual C# é uma das linguagens principais – a gestão de memória é tarefa essencial, que garante a eficiência e a estabilidade das aplicações.
- *Garbage Collection* (GC) é um processo automático que identifica e libera memória não mais acessível ou necessária para o programa.
- O GC rastreia os objetos alocados dinamicamente e determina se eles ainda são alcançáveis. Se um objeto não for mais referenciado por outros objetos ou pelo código, é considerado coletável e sua memória pode ser liberada.
- O GC opera em ciclos e tem várias gerações para otimizar a coleta e minimizar o impacto no desempenho da aplicação.

  - Em linguagens sem GC, como C++, a responsabilidade de alocar e desalocar memória recai sobre o programador, o que pode levar a erros como vazamentos de memória ou acesso a uma memória já liberada.

Disposal

- É uma abordagem complementar ao GC e se refere à liberação explícita de recursos não gerenciados, como conexões de banco de dados, *handles* de arquivos ou recursos de rede.
- Esses recursos não são gerenciados pelo GC e, portanto, não são automaticamente liberados quando o objeto que os detém é coletado.
- A interface IDisposable em .NET fornece um mecanismo para classes gerenciarem a liberação de tais recursos.
- Ao implementar essa interface, as classes fornecem um método Dispose, que pode ser chamado para liberar explicitamente recursos não gerenciados.
    - O uso adequado do método Dispose é crucial para garantir que os recursos sejam liberados em tempo hábil e para evitar bloqueios de recursos ou vazamentos que possam afetar a estabilidade do sistema.
    - Para facilitar seu uso, C# introduziu a instrução *using*, garantindo que o método Dispose seja chamado automaticamente ao final do escopo do objeto.

Debugging

- No desenvolvimento de *software*, é imperativo que desenvolvedores dominem ferramentas e técnicas eficazes para diagnosticar, inspecionar e corrigir problemas em suas aplicações.
- *Debugging*, ou depuração, é o processo pelo qual desenvolvedores identificam e corrigem anomalias ou erros no código-fonte.
- No ambiente .NET, o Visual Studio é frequentemente empregado como a ferramenta principal de *debugging*, oferecendo uma interface rica que permite aos desenvolvedores executar código passo a passo, inspecionar valores de variáveis, avaliar expressões e estabelecer pontos de interrupção, que são locais específicos no código em que a execução será interrompida para inspeção.
    - A depuração é vital não apenas para corrigir erros evidentes, mas também para entender melhor o fluxo de execução e as interações dentro do código, facilitando a identificação de problemas sutis ou comportamentos inesperados.

Tracing

- O *tracing*, ou rastreamento, refere-se ao registro ou monitoramento da execução do programa sem interrompê-lo. É frequentemente utilizado para capturar informações sobre a operação de um aplicativo em tempo real ou em ambientes em que a depuração interativa não é viável, como em produção.
- O *namespace* System.Diagnostics fornece uma série de classes e utilitários que auxiliam no rastreamento, frequentemente realizado ao se gerar *logs* ou mensagens que indiquem a aplicação, pontos de entrada ou saída de métodos, ou situações anômalas.
- A habilidade de ajustar o nível de detalhe desses registros, desde informações básicas até detalhes, permite que os desenvolvedores tenham uma visão granular do comportamento da aplicação em diferentes contextos.

Debugging e *Tracing*

- O *debug* enfrenta condições inesperadas durante o desenvolvimento e o *trace* fornece uma forma contínua de monitorar a operação do programa em diferentes contextos. Ambas as práticas são complementares no desenvolvimento em C#.
- Enquanto a depuração permite uma inspeção profunda e interativa do código durante o desenvolvimento e o teste, o rastreamento fornece *insights* contínuos sobre o comportamento do sistema em ambientes reais, ajudando a identificar problemas que podem não ser evidentes em ambientes de desenvolvimento.
- Dominar as técnicas e ferramentas de *debugging* e *tracing* é essencial para qualquer desenvolvedor C#, pois essas práticas garantem não apenas a correção de erros, mas também a entrega de um *software* robusto, eficiente e confiável para os usuários finais.

Padrões de correspondência

- Podem ser compreendidos como um meio de testar a forma ou o padrão de um dado em vez de simplesmente seu valor, oferecendo uma maneira mais expressiva de trabalhar com dados e possibilitando verificar não só tipos, mas também valores e estruturas internas.
- Na prática, significa que os desenvolvedores podem realizar ações específicas com base em características detalhadas dos dados em vez de apenas seu tipo ou valor superficial.
- Uma de suas vantagens no C# é a capacidade de simplificar o código. Antes da sua introdução, os desenvolvedores, muitas vezes, precisavam escrever várias instruções condicionais aninhadas para avaliar diferentes propriedades e valores de um objeto.
- Com os padrões de correspondência, o processo é mais direto e menos propenso a erros.
    - Outra característica valiosa é a integração deles com outras funcionalidades, como as expressões de *switch* e as tuplas.
    - É possível usá-los para identificar padrões específicos, mas também para destrinchar dados em componentes mais gerenciáveis, que podem ser processados ou transformados de acordo com as necessidades do desenvolvedor.

Padrões de correspondência

- Alguns padrões.

Padrão	Descrição
Tipo e Var Patterns	Permitem que os programadores verifiquem se um objeto é de determinado tipo; se for, o objeto pode ser automaticamente convertido para esse tipo, eliminando a necessidade de conversões explícitas
Constant Pattern	Permite verificar se uma variável corresponde a um valor constante específico
Recursive Patterns	Funcionalidade avançada que possibilita verificar propriedades e campos internos de um objeto. Por exemplo, pode verificar se um objeto contém certas propriedades com valores específicos
Property Pattern	Semelhante ao Recursive Pattern, foca a verificação das propriedades de um objeto
Tuple Pattern	Ideal para trabalhar com tuplas, permite que os desenvolvedores verifiquem múltiplos valores simultaneamente
Positional Pattern	Forma frequentemente usada com tipos que têm uma ordem ou posição específica para seus membros, como registros ou tuplas

Fonte: autoria própria.

Interatividade

Qual das afirmações a seguir está correta?

a) Expressões lambda e LINQ não têm aplicabilidade em *debugging* e *disposal* de recursos.

b) *Debug* é um processo automatizado que utiliza expressões lambda para identificar erros.

c) LINQ é uma ferramenta de *tracing* que registra operações de *garbage collection*.

d) O *garbage collection* em C# é manualmente controlado pelos programadores por expressões lambda e LINQ para otimizar a *performance*.

e) Expressões lambda e LINQ são frequentemente usadas para escrever consultas de dados mais concisas e legíveis, enquanto *debugging* e *tracing* são utilizados para monitoramento e diagnóstico de aplicativos, e o *garbage collection* automático ajuda na gestão de memória.

Resposta

Qual das afirmações a seguir está correta?

a) Expressões lambda e LINQ não têm aplicabilidade em *debugging* e *disposal* de recursos.

b) *Debug* é um processo automatizado que utiliza expressões lambda para identificar erros.

c) LINQ é uma ferramenta de *tracing* que registra operações de *garbage collection*.

d) O *garbage collection* em C# é manualmente controlado pelos programadores por expressões lambda e LINQ para otimizar a *performance*.

e) Expressões lambda e LINQ são frequentemente usadas para escrever consultas de dados mais concisas e legíveis, enquanto *debugging* e *tracing* são utilizados para monitoramento e diagnóstico de aplicativos, e o *garbage collection* automático ajuda na gestão de memória.

Threading

- Refere-se ao conceito de execução simultânea de múltiplas tarefas dentro de uma aplicação, em que cada tarefa é tratada como uma *thread* (ou linha de execução).

- O suporte a esse conceito é majoritariamente fornecido pelo *namespace* System.Threading.

- No contexto de um programa, *thread* é a menor unidade de um processo que pode operar de forma independente. Um único processo pode conter várias *threads*, o que permite a execução paralela de tarefas, especialmente em sistemas com múltiplos núcleos de CPU.

  - A classe *Thread* é uma das principais ferramentas que permitem criar e gerenciar *threads* para evitar o custo de criá-las e destruí-las manualmente, o C# oferece o ThreadPool, um conjunto de *threads* pré-inicializadas prontas para executar tarefas.

Threading – desafios reais

- Quando vários clientes tentam retirar dinheiro da mesma conta ao mesmo tempo, o sistema deve garantir que o saldo da conta seja atualizado corretamente para evitar que mais dinheiro seja retirado do que o disponível.
- Quando um sistema tenta realizar transferências entre duas contas em ordens diferentes (conta A para conta B e vice-versa) e cada operação exige bloqueios exclusivos nas contas envolvidas, pode haver um impasse se as operações forem simultâneas e cada uma esperar a outra liberar o bloqueio.
- Quando vários clientes tentam reservar o último assento disponível em um voo, o sistema precisa garantir que apenas um cliente consiga a reserva, enquanto os outros recebem uma mensagem informando que o assento não está mais disponível.
- Durante promoções ou lançamentos de produtos populares, muitos usuários podem tentar comprar o mesmo item ao mesmo tempo. O sistema deve gerenciar o estoque corretamente, garantindo que os pedidos sejam processados com base na disponibilidade e evitando vender mais itens do que o disponível em estoque.

Threading – desafios reais

- Em ambientes de trabalho colaborativo, como Google Docs ou Microsoft Office Online, vários usuários podem editar um documento simultaneamente. O sistema deve garantir que as edições de um usuário não sobrescrevam indevidamente as de outro, mantendo o documento consistente para todos.
- Em jogos de mundo aberto, em que vários jogadores podem interagir com os mesmos objetos ou NPCs (personagens não jogáveis), o servidor do jogo precisa sincronizar as ações para garantir que todos os jogadores vejam o mesmo estado do mundo em tempo real.
- Em ambientes como hospitais ou departamentos de atendimento ao cliente, em que pessoas são atendidas com base em sua ordem de chegada ou prioridade, o sistema deve garantir que cada pessoa ou requisição seja atendida de forma justa e ordenada.
  - Em todos os exemplos, é crucial que os sistemas sejam projetados para lidar com acessos concorrentes de maneira apropriada, garantindo a integridade dos dados, a justiça e uma experiência de usuário eficiente. Falhar em administrar adequadamente esses desafios pode resultar em erros, perda de dados ou insatisfação do cliente.

Threading – detalhes técnicos

- Em sistemas *multithreaded*, a execução exata pode variar dependendo do agendamento do sistema operacional e de outros fatores. No entanto, devido aos bloqueios (*lock*) implementados no código, a ordem das mensagens dentro de cada função é garantida.
- Ferramentas como Mutex (*Mutual Exclusion*) garantem que apenas uma *thread* tenha acesso a recursos específicos ou seções de código de cada vez. É um mecanismo de sincronização que garante a apenas uma *thread* acessar recursos ou seções de código específicos de cada vez.
- Da mesma forma, os conceitos de Semaphore, Monitor e lock limitam ou sincronizam o acesso de *threads* a determinadas partes do código ou recursos.
    - Semaphore permite limitar o número de *threads* que podem acessar um recurso ou seção de código simultaneamente.
    - Monitor e lock garantem que apenas uma *thread* execute um bloco de código específico de cada vez.
    - ManualResetEvent e AutoResetEvent ajudam a sinalização entre *threads*, garantindo que uma ou mais *threads* esperem até que recebam sinal para continuar a execução.

Task

- É uma unidade de trabalho que pode ser executada de forma assíncrona. A biblioteca de tarefas paralelas, conhecida como *Task Parallel Library* (TPL), introduziu o conceito de tarefa como peça fundamental para a programação assíncrona e paralela.
- A programação assíncrona permite que a aplicação inicie uma operação – como a solicitação ao servidor – e continue executando outras tarefas enquanto espera a conclusão. Quando a operação assíncrona é concluída, a aplicação é notificada e pode processar os resultados, tudo isso sem ter bloqueado a execução principal da aplicação.
- A programação paralela se relaciona à execução simultânea de múltiplas operações. Em sistemas modernos, com múltiplos núcleos de processador, é possível executar várias tarefas ao mesmo tempo, literalmente em paralelo.
    - Por exemplo, se tivéssemos um grande conjunto de dados a processar, poderíamos dividi-lo em partes menores e processar várias partes simultaneamente, em diferentes núcleos. A programação paralela permite que as aplicações tirem proveito dessa capacidade, acelerando operações que seriam demoradas se executadas sequencialmente.

Task vs Thread

- A TLP fornece ferramentas e estruturas para facilitar tanto a programação assíncrona quanto a paralela; com ela, os desenvolvedores podem iniciar tarefas assíncronas, aguardar sua conclusão e paralelizar operações, como *loops*, para que sejam executadas simultaneamente em múltiplos núcleos.
- Tanto *threads* quanto *tasks* lidam com a execução concorrente de código, mas abordam essa concorrência de maneiras diferentes e servem a propósitos distintos.
- A classe *Thread* representa uma execução singular e independente no nível do sistema operacional. Ao criar e iniciar uma nova instância de *Thread*, estamos literalmente solicitando que o sistema operacional aloque recursos para uma nova *thread* de execução.
    - O desenvolvedor tem um controle granular sobre a execução, mas, com esse controle, vem a responsabilidade de gerenciar explicitamente todos os aspectos dela, como sincronização e possíveis conflitos dessa *thread.*

Task vs Thread

- A *Task* é uma abstração de alto nível sobre a execução assíncrona e, muitas vezes, pode ser vista como representação de uma operação ocorrendo em segundo plano.
- Uma *Task* não é uma *thread* em si; em vez disso, a TPL utiliza um *pool* de *threads* para executar *tasks.*
- *Pool* de *threads* é uma coleção pré-inicializada de *threads* prontas para executar tarefas, sem a necessidade de criar novas *threads* do zero a cada vez que uma operação concorrente é necessária.
- Ao criar e iniciar uma *Task*, ela será agendada para execução em uma das *threads* desse *pool*, sem a necessidade de criar explicitamente uma nova *thread* do sistema operacional.
    - Reutilizar *threads* de um *pool* é geralmente mais eficiente do que criar e destruir constantemente novas *threads.*
    - A abstração fornecida pela *Task* permite uma programação assíncrona mais fácil e intuitiva.
    - A TPL se encarrega de muitos detalhes de gerenciamento e sincronização, permitindo que os desenvolvedores se concentrem na lógica do aplicativo.

Task vs Thread

- Enquanto *threads* são sobre "como" o código é executado paralelamente (os detalhes da execução), *tasks* são mais sobre "o que" está sendo executado, com um foco em operações de alto nível, muitas vezes assíncronas.

Threads **(“Como”)**	***Tasks*** **(“O que”)**
▪ São unidades de execução de baixo nível. ▪ *Threads* operam dentro do sistema operacional, e cada *thread* pode ser pensada como um caminho separado de execução no código. ▪ Elas são úteis para operações que requerem controle fino sobre a execução paralela ou concorrente. ▪ No entanto, gerenciar *threads* diretamente pode ser complexo, especialmente em relação à sincronização e ao compartilhamento de recursos.	▪ São abstrações de mais alto nível, geralmente usadas para operações assíncronas. ▪ Uma *task* em C# representa uma operação que pode ou não estar executando em uma *thread* separada. ▪ *Tasks* são parte da TPL (*Task Parallel Library*) e oferecem uma maneira mais gerenciável e escalável de lidar com a execução paralela e assíncrona. ▪ Elas permitem que você escreva código assíncrono que é mais fácil de ler, escrever e manter, abstraindo muitos dos detalhes complexos do gerenciamento de *threads*.

Task

- O tipo *Task* é a representação principal de uma unidade de trabalho que pode estar em andamento ou já concluída.
- Quando essa unidade de trabalho é projetada para retornar um valor após sua conclusão, utilizamos Task<T>, em que T especifica o tipo do valor a ser retornado.
- Dois modificadores-chave: *async* e *await*. O *async* permite definir métodos que retornarão *tasks*; e, dentro desses métodos, a palavra-chave *await* chama outros métodos que retornam *tasks*, pausando temporariamente a execução do método atual até que a *task* invocada seja concluída, mas sem bloquear a *thread* em que está sendo executada.
  - Iniciar uma *task* é uma operação simples, geralmente realizada usando o método Task.Run(), que pega um delegado e o executa em uma *thread* separada.
  - Quando uma *task* é finalizada, é importante considerar o tratamento adequado, especialmente porque ela pode lançar exceções que, se lançadas dentro de uma tarefa, serão retransmitidas ao tentarmos acessar o resultado da *task* ou explicitamente esperar sua conclusão.

Task

- Um aspecto crucial das tarefas em C# é a capacidade de cancelá-las. O cancelamento é implementado pela classe CancellationTokenSource, que fornece um *token* de cancelamento.
- O código executado dentro da *task* pode então verificar periodicamente esse *token* para verificar se um cancelamento foi pedido, permitindo um encerramento simples e objetivo.
- Concluída uma *task*, pode-se desejar que outro bloco de código seja executado imediatamente depois. Isso é gerenciado usando o método ContinueWith, que permite encadear tarefas, garantindo que certo código seja executado após a conclusão de uma tarefa específica.
    - O C# 7.1 introduziu o suporte para um método Main assíncrono, permitindo que os desenvolvedores definissem o Main como async Task ou async Task <int>, facilitando a chamada de métodos assíncronos diretamente do Main usando o *await*, sem a necessidade de bloquear a *thread* principal.

Funções e padrões assíncronos – EAP

- O *Event-Based Asynchronous Pattern* (EAP) é uma abordagem orientada a eventos para operações assíncronas.
- Esse padrão depende de eventos para notificar o chamador sobre o início, progresso e conclusão de uma operação assíncrona.
- Os componentes que utilizam EAP geralmente adotam um método Start para iniciar a operação assíncrona e um evento Completed para indicar que ela terminou.
- Podem incluir eventos como ProgressChanged para fornecer atualizações intermediárias.
- O EAP oferece uma maneira de cancelar operações em andamento por meio de métodos como Cancel.

  - Ainda que o EAP torne certas operações mais intuitivas – principalmente para desenvolvedores familiarizados com a programação orientada a eventos –, pode ser desafiador gerenciar múltiplas operações assíncronas simultâneas.

Funções e padrões assíncronos – APM

- O *Asynchronous Programming Model* (APM) é frequentemente reconhecido por sua abordagem de pares Begin e End.
- Em vez de depender de eventos, utiliza *callbacks* para notificar o término de operações.
- Abordagem baseada em *callbacks* refere-se a um padrão de projeto no qual uma função é passada como argumento (ou parâmetro) para outra função, para ser executada em um momento posterior.
- Esse retorno de chamada (ou *callback*) geralmente é invocado após a conclusão de uma operação assíncrona, permitindo que o código continue a execução enquanto aguarda o resultado de uma tarefa demorada – como leitura de arquivos, solicitações de rede etc.
    - A ideia é que uma operação assíncrona seja iniciada com um método que tenha o prefixo Begin e seja concluída com um método de prefixo End.
    - O método Begin normalmente aceita um *delegate* de *callback* e um objeto de estado, passado para o método End quando a operação é concluída.

Funções e padrões assíncronos

- Embora o APM forneça um grande controle sobre as operações assíncronas, pode se tornar complexo rapidamente, especialmente ao encadear várias operações assíncronas.

Ambos os padrões, EAP e APM, têm seus méritos:

- O EAP, com sua orientação a eventos, pode ser mais intuitivo para certos cenários, especialmente quando as atualizações intermediárias são essenciais.
- Já o APM, com sua abordagem baseada em *callbacks*, oferece uma granularidade que pode ser benéfica em situações que exigem controle meticuloso. No entanto, com a evolução do C# e a introdução das palavras-chave *async* e *await*, a programação assíncrona tornou-se mais acessível e direta.

- Apesar desses novos paradigmas, é essencial reconhecer e entender EAP e APM, pois eles ainda existem em muitos <u>sistemas legados</u> e fornecem uma base para apreciar as inovações mais recentes em programação assíncrona em C#.

Interatividade

Qual das seguintes afirmações é correta sobre *Tasks*, *Threads*, APM e EAP em C#?

a) *Tasks* e *Threads* são conceitos idênticos, ambos representando unidades de trabalho em um único núcleo do processador.

b) O modelo APM em C# é uma técnica moderna de programação assíncrona que utiliza *Tasks*, enquanto o EAP é baseado em *Threads* e é considerado menos eficiente.

c) *Tasks* e *Threads* são usados para operações de programação assíncrona e concorrente, em que *Tasks* são mais abstratas e baseadas em *Threads*.

d) EAP é um modelo de *threading* que permite a execução de múltiplas *Tasks*.

e) APM e EAP são tecnologias de coleta de lixo e gerenciamento de memória.

Resposta

Qual das seguintes afirmações é correta sobre *Tasks*, *Threads*, APM e EAP em C#?

a) *Tasks* e *Threads* são conceitos idênticos, ambos representando unidades de trabalho em um único núcleo do processador.

b) O modelo APM em C# é uma técnica moderna de programação assíncrona que utiliza *Tasks*, enquanto o EAP é baseado em *Threads* e é considerado menos eficiente.

c) *Tasks* e *Threads* são usados para operações de programação assíncrona e concorrente, em que *Tasks* são mais abstratas e baseadas em *Threads*.

d) EAP é um modelo de *threading* que permite a execução de múltiplas *Tasks*.

e) APM e EAP são tecnologias de coleta de lixo e gerenciamento de memória.

Classes e algoritmos

- Uma característica distintiva do C# é sua extensa biblioteca-padrão, pois oferece múltiplas estruturas que auxiliam desenvolvedores a manipular e organizar dados.
- Em relação às que tratam dados de maneira sequencial, o C# oferece recursos que facilitam a inserção, remoção e busca de informações, sendo fundamentais em ambientes de desenvolvimento em que a sequência e a estrutura dos dados desempenham um papel crucial na lógica do programa.
- Com eles, os desenvolvedores podem construir sistemas que lidam com volumes significativos de dados, otimizando o tempo de desenvolvimento e a manutenção de programas.
  - No âmbito da recuperação rápida de dados, o C# é equipado com estruturas otimizadas, eficazes para associar dados e verificar sua presença.
  - Essas estruturas permitem que os desenvolvedores realizem operações de forma ágil e precisa, independentemente do tamanho ou complexidade dos conjuntos de dados em questão.

Ordenação – vetores e listas

- É uma atividade fundamental na programação e na ciência da computação, e diversos algoritmos foram desenvolvidos ao longo dos anos para isso.
- O algoritmo de Seleção (*selection sort*) opera buscando, iterativamente, o menor (ou maior) elemento do vetor ou lista e colocando-o na posição correta (Cormen *et al*., 2002), mas não é o algoritmo mais eficiente para grandes volumes de dados.
- O Inserção (*Insertion Sort*) é outro algoritmo simples, mas que, sob determinadas circunstâncias, pode ser mais eficaz. A ideia aqui é, a cada iteração, pegar um elemento e inseri-lo na posição correta entre os elementos anteriormente analisados (Ziviani, 2004).
- O *Bubble Sort* é talvez um dos algoritmos de ordenação mais conhecidos, embora não seja dos mais eficientes.
    - Ele trabalha repetidamente percorrendo a lista, comparando pares de elementos adjacentes e trocando-os se estiverem na ordem errada.
    - O processo se repete até que nenhuma troca seja necessária, indicando que a lista está ordenada (Sedgewick, 2013).

Ordenação – vetores e listas

- O Quick Sort é um algoritmo de ordenação mais avançado e eficiente para conjuntos maiores de dados (Cormen *et al*., 2002), baseando-se em uma técnica de divisão e conquista.

- O processo envolve a escolha de um “pivô” e, em seguida, a divisão da lista em duas partes: uma com elementos menores que o pivô e outra com elementos maiores; e o algoritmo é então aplicado recursivamente a essas duas sublistas.

- Quando bem implementado, o Quick Sort pode ser significativamente mais rápido que os algoritmos mencionados, especialmente para listas grandes.

  - Esses algoritmos podem ser implementados utilizando estruturas de dados nativas, como *arrays* e *List*, bem como operações básicas de iteração e comparação.

Ordenação – vetores e listas

- Aplicações práticas.

Selection Sort

- Em sistemas simples de gerenciamento de bibliotecas, em que o acervo é limitado e a frequência de novas adições e pesquisas é baixa, o Selection Sort pode ordenar livros por título ou autor, dada a simplicidade do algoritmo e a ausência de necessidade de alto desempenho.

Insertion Sort ou Bubble Sort

- Em um jogo digital cujos envolvidos recebem cartas de forma sequencial e precisam mantê-las ordenadas na mão. Como elas são recebidas uma de cada vez, e a mão do jogador geralmente é pequena, o Insertion Sort é eficiente para inserir cada nova carta na posição correta. Em sistemas didáticos ou educativos – cujo objetivo é ensinar os fundamentos da ordenação e cuja visualização do processo é mais importante que a eficiência –, o Bubble Sort pode ser utilizado, dado que sua lógica é fácil de entender e visualizar.

Quick Sort

- Em um *e-commerce* com milhares de produtos, ao se fazer uma pesquisa, os resultados precisam ser ordenados por relevância, preço ou avaliação do cliente rapidamente. Dado o grande volume de produtos e a necessidade de respostas rápidas, o Quick Sort pode ser uma excelente escolha, pois é eficiente em grandes conjuntos de dados e geralmente supera os algoritmos de ordenação.

Pilhas e filas

- Pilhas e filas são estruturas de dados fundamentais, que armazenam e gerenciam informações em uma ordem específica.

- Ambas têm aplicações em diversos domínios, desde o processamento de dados até operações em sistemas e aplicações de *software.*

- Uma pilha, como o nome sugere, pode ser visualizada como uma pilha de pratos. Imagine que você está empilhando pratos um sobre o outro. O prato que você coloca por último é o primeiro que você tira quando começa a desempilhar – comportamento conhecido como *last in*, *first out* (Lifo), ou seja, o último item que entra é o primeiro a sair.

  - Em termos técnicos, uma pilha permite duas operações principais: *push*, que adiciona um elemento no topo da pilha; e *pop*, que remove o elemento do topo.

Pilhas

- Pilhas são comumente usadas em operações como a avaliação de expressões matemáticas e o rastreamento de chamadas de funções em linguagens de programação.

- Uma das aplicações mais intuitivas da pilha é o recurso “desfazer” (ou “undo”), presente em muitos *softwares*, desde editores de texto até *softwares* de *design* gráfico. Quando você executa uma série de ações – como digitar, apagar ou desenhar –, elas são empilhadas. Se você decide desfazer sua ação mais recente, a operação no topo da pilha é revertida. Quanto mais vezes você pressiona “desfazer”, mais ações são desempilhadas e revertidas em ordem.

- Da mesma forma, muitos navegadores de internet utilizam pilhas para gerenciar o histórico de navegação. Quando você clica no botão “voltar”, é para o *site* no topo da pilha é que você retorna.

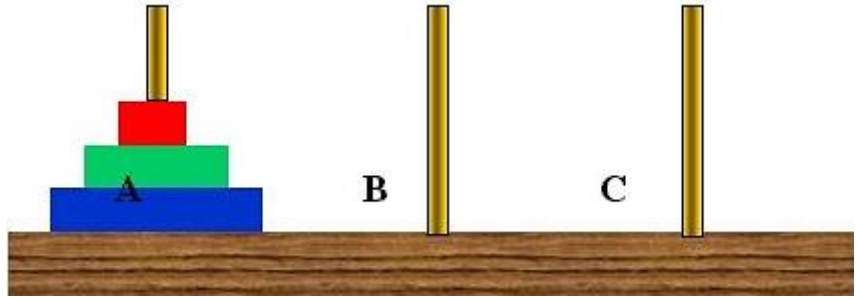
Torre de Hanói – o jogo

- Torre de Hanói é um clássico problema de quebra-cabeça que se baseia no uso de pilhas.
- Consiste em três hastes (ou pinos) e um número de discos de tamanhos diferentes, que podem deslizar para qualquer haste. O quebra-cabeça começa com os discos numa pilha de ordem crescente numa haste (com o menor no topo), formando uma espécie de pirâmide.

O objetivo do quebra-cabeça é mover a pilha inteira para outra haste, obedecendo às seguintes regras:

- Apenas um disco pode ser movido de cada vez.
- Cada movimento consiste em pegar o disco superior de uma das hastes e deslizá-lo para o topo de outra haste.
- Um disco não pode ser colocado em cima de um disco menor.

Fonte: https://commons.wikimedia.org/wiki/File:Torre_de_Han%C3%B3i.jpg



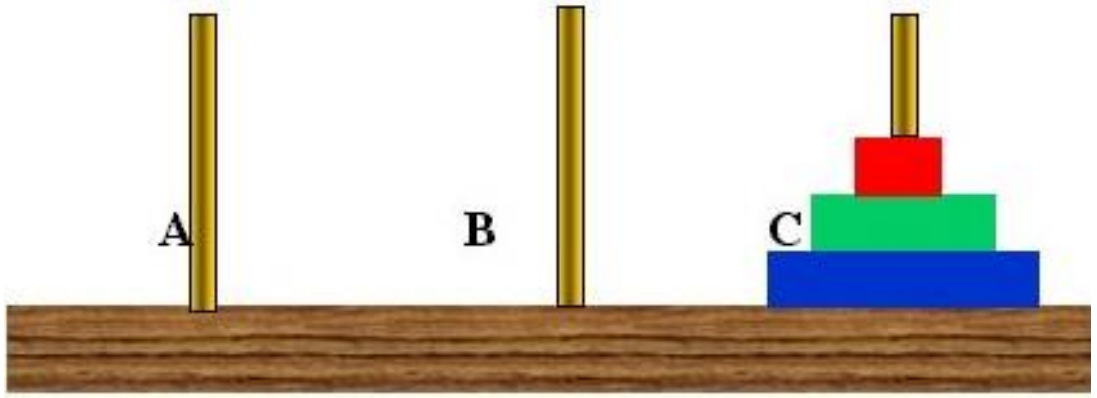
Torre de Hanói – recursão

- Do ponto de vista computacional, a forma mais elegante de resolver o quebra-cabeça é um algoritmo recursivo.
- Recursão é um conceito fundamental em ciência da computação e matemática, e refere-se ao processo em que uma função chama a si mesma em sua definição.
- Na programação, uma função recursiva resolve uma tarefa em partes, invocando a mesma função com argumentos diferentes, até que a solução seja alcançada.
- Essa abordagem é especialmente útil para problemas que podem ser divididos em subproblemas menores, de natureza similar ao problema original.
    - Uma característica importante de uma função recursiva é a presença de uma condição de encerramento, também conhecida como caso-base.
    - Sem essa condição, a função continuaria a chamar a si mesma indefinidamente, levando a um ciclo infinito.
    - Caso-base é o ponto em que a função retorna um valor sem fazer outra chamada recursiva.

Torre de Hanói – a solução recursiva

- A solução para a Torre de Hanói é frequentemente implementada com uma abordagem recursiva e, durante esse processo, a estrutura de pilha é intrínseca ao problema.
- Cada haste pode ser vista como pilha, em que os discos são empurrados e desempilhados conforme as regras do jogo.
- Ao mover os discos entre as hastes, estamos essencialmente realizando operações de *push* e *pop* em pilhas.
- Se quisermos mover n discos de uma haste origem para uma haste destino, podemos começar movendo n-1 discos para uma haste auxiliar, mover o disco n para a haste destino e depois mover os n-1 discos da haste auxiliar para a haste destino.

Fonte: https://commons.wikimedia.org/wiki/File:Torre_de_Han%C3%B3i.jpg



Fila

- Pode ser comparada a uma fila comum, de pessoas esperando num banco, por exemplo.
- A primeira pessoa que entra na fila é a primeira a ser atendida e a sair dela – comportamento conhecido como *first in, first out* (Fifo).
- Assim, numa fila temos operações principais, como *enqueue* – que adiciona um elemento ao final dela – e *dequeue* – que remove o elemento do início.
- Filas envolvem gestão de tarefas em sequência, como em sistemas de impressão, em que o primeiro documento enviado para a impressora é o primeiro a ser impresso. Se vários usuários enviarem documentos para uma impressora ao mesmo tempo, a impressora os enfileira e os imprime em ordem, baseando-se no Fifo.
    - Quando um *e-mail* é enviado, ele não é transmitido imediatamente para o destinatário; em vez disso, é colocado numa fila de mensagens a enviar.
    - O servidor de *e-mail* processa essa fila, enviando uma mensagem de cada vez, na ordem em que foram recebidas.

Pilhas e filas

- Tanto pilhas quanto filas são facilmente implementadas, respectivamente, por meio das classes Stack e Queue, disponíveis na biblioteca-padrão .NET.
- Elas oferecem métodos e propriedades que facilitam a manipulação e o gerenciamento de dados de acordo com as características inerentes a cada estrutura.
- Pilhas e filas são mais do que meras abstrações da ciência da computação; estão incorporadas em muitos sistemas e aplicativos que usamos no dia a dia, embora possamos não as perceber diretamente.
- Entender o funcionamento dessas estruturas é crucial para qualquer desenvolvedor, pois fornecem mecanismos para gerenciar dados de maneira ordenada e previsível.

Dicionários e conjuntos

- São estruturas de dados que armazenam elementos de forma eficiente, permitindo operações rápidas de busca, inserção e remoção.
- Essas estruturas são implementadas usando uma técnica chamada tabelas de *hash.*
- Um dicionário, especificamente, armazena pares de chave e valor, que num dicionário são únicas, e cada chave está associada a um valor específico. Isso permite que os usuários acessem um valor rapidamente, fornecendo sua chave correspondente.
- Um conjunto, ou *set*, armazena valores únicos, sem associações de chave-valor.
- Ambos, dicionários e conjuntos, aproveitam a técnica de tabelas de *hash* para alcançar eficiência.
  - Essas tabelas trabalham convertendo a chave (ou valor, no caso de conjuntos) em um índice de *array* usando uma função *hash.*
  - Esse índice determina em que o valor associado será armazenado na tabela.

Dicionários e conjuntos

- O tempo necessário para encontrar um item numa tabela não depende diretamente do número de elementos armazenados na estrutura.
- É importante observar que a eficiência de uma tabela depende de uma boa função *hash* e da capacidade de lidar com colisões (quando duas chaves diferentes resultam no mesmo índice).
- Considere um serviço de *streaming* de música como o Spotify. Quando busca uma música ou artista, o sistema precisa encontrar essa informação rapidamente em meio a milhões de músicas e artistas disponíveis. A busca eficiente em grandes volumes de dados pode ser viabilizada por dicionários, em que a chave pode ser o nome da música ou do artista, e o valor associado é a informação detalhada ou o local onde a música está armazenada.
    - Nas redes sociais, ao recebermos sugestões de “Pessoas que talvez conheça”, o sistema, por trás das cenas, pode usar conjuntos para armazenar e processar rapidamente ID de usuários dos quais já somos amigos, garantindo que as sugestões não incluam pessoas que já fazem parte da nossa lista.

Interatividade

Qual das seguintes afirmações é correta sobre ordenação, filas, pilhas e dicionários?

a) Dicionários são automaticamente ordenados, enquanto vetores, filas e pilhas não são.
b) A ordenação de vetores, pilhas e filas é realizada usando o método Sort().
c) Pilhas são usadas para ordenação de dados, operando com o princípio FIFO (*First In, First Out*), assim como as filas.
d) Os vetores podem ser ordenados usando algoritmos de ordenação personalizados, filas operam com o princípio FIFO, pilhas com o princípio LIFO e dicionários não têm ordenação.
e) Filas, pilhas e dicionários são estruturas de dados que possuem métodos integrados para ordenação, semelhantes aos vetores.

Resposta

Qual das seguintes afirmações é correta sobre ordenação, filas, pilhas e dicionários?

a) Dicionários são automaticamente ordenados, enquanto vetores, filas e pilhas não são.
b) A ordenação de vetores, pilhas e filas é realizada usando o método Sort().
c) Pilhas são usadas para ordenação de dados, operando com o princípio FIFO (*First In, First Out*), assim como as filas.
d) Os vetores podem ser ordenados usando algoritmos de ordenação personalizados, filas operam com o princípio FIFO, pilhas com o princípio LIFO e dicionários não têm ordenação.
e) Filas, pilhas e dicionários são estruturas de dados que possuem métodos integrados para ordenação, semelhantes aos vetores.

Dicionários e conjuntos

- Existem classes específicas projetadas para implementar dicionários e conjuntos, proporcionando aos desenvolvedores as ferramentas necessárias para armazenar e gerenciar dados de maneira eficaz e eficiente.

- Dictionary<TKey, TValue>, que faz parte do *namespace* System.Collections.Generic permite que os desenvolvedores armazenem pares de chave-valor.

- Com ela, é possível associar uma chave única a um valor específico, e essa associação garante que os dados sejam recuperados de maneira rápida, aproveitando as características subjacentes das tabelas de *hash.*

  - Esse dicionário é especialmente útil quando a busca rápida e a recuperação de dados são essenciais, como em sistemas de cache ou quando se deseja mapear identificadores a objetos complexos.

Dicionários e conjuntos

- A classe HashSet<T> também é parte do *namespace* System.Collections.Generic.

- A principal característica dos conjuntos é que eles armazenam itens únicos, ou seja, não permitem duplicatas, tornando-os ideais quando se deseja manter uma coleção de itens distintos, como uma lista de IDs de usuários únicos.

- A classe HashSet garante que os itens sejam armazenados e acessados de forma eficiente, também aproveitando os princípios das tabelas de *hash.*

  - Ambas as classes, Dictionary e HashSet, são representantes claros de como o C# encapsula e oferece implementações de alto nível de estruturas de dados complexas, que permitem aos desenvolvedores se beneficiar da eficiência e eficácia das tabelas de *hash.*

Árvores

- São uma das abordagens mais interessantes para organizar informações de forma hierárquica e têm uma presença marcante, especialmente quando se trata de armazenamento e recuperação eficientes de dados.
- Em uma árvore binária, cada nó tem no máximo dois filhos, muitas vezes denominados subárvore esquerda e subárvore direita. Uma característica importante de algumas árvores binárias é a forma como são organizadas.
- Quando se tem uma árvore de busca binária, por exemplo, os valores são dispostos de tal forma que, para qualquer nó dado, os valores à esquerda são menores que o valor do nó, e os valores à direita são maiores. Essa disposição permite uma busca eficiente, pois elimina metade das opções possíveis a cada passo.
    - Uma preocupação ao usar árvore de busca binária é seu equilíbrio. Se os dados forem inseridos em uma ordem particular, ela pode ficar muito inclinada, assemelhando-se mais a uma lista vinculada. Essa situação elimina os benefícios de tempo de busca da estrutura de árvore.
    - É aqui que entram as árvores autobalanceadas.

Árvores

- As árvores autobalanceadas são árvores de busca binária que se reorganizam durante as operações de inserção e remoção para garantir que a árvore permaneça equilibrada, otimizando assim o tempo de busca.

- A biblioteca-padrão não fornece uma implementação direta dessas árvores, mas estruturas como SortedSet<T> e SortedDictionary<TKey, TValue> se baseiam em árvores e oferecem características de desempenho semelhantes.

  - Essas classes encapsulam a complexidade das operações da árvore, permitindo que os desenvolvedores se beneficiem de suas vantagens sem se aprofundar nos detalhes de sua implementação.

Heaps

- Desempenham papel fundamental em muitos algoritmos e aplicações.

- São um tipo especial de árvore binária (ou, em alguns casos, não binária) que segue determinadas propriedades. O *heap* binário é, talvez, o mais familiar de todos. Trata-se de uma árvore binária completa, em que cada nó tem um valor que respeita uma relação específica se comparado aos valores.

  - Existem dois tipos principais de *heaps* binários: *max-heaps* e *min-heaps*. Em um *max-heap*, o valor de cada nó é sempre maior ou igual aos valores de seus filhos, enquanto em um *min-heap* é sempre menor ou igual. Isso permite operações eficientes como inserção, exclusão e, mais notavelmente, extração do elemento máximo ou mínimo de seus nós filhos.

Heaps

- Os *heaps* binomiais são uma extensão interessante da ideia dos *heaps* binários – uma coleção de árvores binomiais, em que cada árvore é um *heap.*

- A peculiaridade dos *heaps* binomiais é permitir operações como união de dois *heaps* de maneira eficiente, num tipo de *heap* formado pela combinação de outros *heaps* binomiais menores, seguindo propriedades específicas que garantem sua eficiência.

- Os *heaps* de Fibonacci são uma coleção de árvores que não necessariamente precisam ser binárias e que são muito hábeis em certas operações, como diminuir uma chave ou excluir um nó em tempo amortizado constante.

  - Essa eficiência torna-as uma escolha atrativa para algoritmos que necessitam de operações prioritárias rápidas, como o algoritmo de Dijkstra para caminhos mais curtos (Dijkstra, 1959), uma abordagem fundamental na teoria dos grafos para encontrar o caminho mais curto entre dois nós em um grafo ponderado.

Grafos

- São estruturas de dados que podem representar relações binárias entre um conjunto de objetos e são formados por vértices (nós), conectados por arestas ou arcos.
- São amplamente usados para resolver problemas e modelar sistemas em campos variados, como redes de computadores, sistemas de transporte e redes sociais.
- Em essência, são instrumentos matemáticos extremamente valiosos e versáteis, aplicáveis em diversas situações cotidianas e profissionais, servindo como base para modelar e resolver uma variedade de problemas práticos em diferentes domínios.
- Frequentemente, representam relações entre objetos ou entidades, proporcionando uma representação visual e estrutural de sistemas complexos.

Grafos

- O algoritmo de Kruskal é uma técnica que encontra a árvore geradora mínima em um grafo (Kruskal, 1956), ou seja, um subconjunto do grafo que conecta todos os vértices com o menor custo total. Ele começa tratando cada vértice como componente separado e, iterativamente, seleciona a aresta de menor peso que conecta dois componentes distintos, continuando esse processo até que todos os vértices estejam conectados.
- O algoritmo de Prim, por outro lado, também encontra a árvore geradora mínima em um grafo, mas adota uma abordagem diferente do Kruskal; ele começa a partir de um vértice arbitrário e, a cada passo, adiciona ao conjunto a aresta de menor peso que conecta um vértice dentro do conjunto a um vértice fora dele, continuando o processo até que todos os vértices estejam no conjunto.
- O algoritmo de Dijkstra encontra o caminho mais curto entre dois vértices em um grafo ponderado, em que arestas têm pesos associados a elas, que podem representar, por exemplo, distâncias ou custos. Esse algoritmo trabalha de maneira incremental, começando do vértice de origem e expandindo-se para os vértices vizinhos, atualizando as distâncias conhecidas, até alcançar o vértice de destino.

Grafos

- Um grafo pode ser direcionado ou não direcionado, ponderado ou não ponderado.

- Em um grafo direcionado, cada aresta tem uma direção, indo de um vértice origem para um vértice destino. Em um grafo não direcionado, as arestas não têm direção.

- Grafos ponderados têm valor ou peso associado a cada aresta, representando, por exemplo, a distância entre dois pontos ou o custo para percorrer uma conexão. Grafos não ponderados não têm pesos associados às suas arestas.

  - Uma abordagem comum é criar uma classe para representar o grafo, uma classe para os vértices e uma classe para as arestas. Estruturas de dados, como listas e dicionários, podem armazenar vértices e arestas, e manter informações sobre as relações entre eles.

Interatividade

Qual das seguintes afirmações é correta sobre a implementação de árvores, *heaps* e grafos?

a) A biblioteca-padrão inclui implementações diretas de árvores, *heaps* e grafos.

b) *Heaps* são um tipo de estrutura de dados que C# trata como *arrays* com propriedades especiais, enquanto grafos não podem ser implementados em C#.

c) Árvores, especialmente árvores binárias, e *heaps*, como o *heap* binário, são comumente implementados em C# para algoritmos de ordenação e filas de prioridade.

d) C# fornece uma implementação nativa de grafos por meio da biblioteca System.Graph.

e) Na programação C#, grafos são geralmente implementados como tipos de referência, enquanto árvores e *heaps* são considerados tipos de valor.

Resposta

Qual das seguintes afirmações é correta sobre a implementação de árvores, *heaps* e grafos?

a) A biblioteca-padrão inclui implementações diretas de árvores, *heaps* e grafos.

b) *Heaps* são um tipo de estrutura de dados que C# trata como *arrays* com propriedades especiais, enquanto grafos não podem ser implementados em C#.

c) Árvores, especialmente árvores binárias, e *heaps*, como o *heap* binário, são comumente implementados em C# para algoritmos de ordenação e filas de prioridade.

d) C# fornece uma implementação nativa de grafos por meio da biblioteca System.Graph.

e) Na programação C#, grafos são geralmente implementados como tipos de referência, enquanto árvores e *heaps* são considerados tipos de valor.

Referências

- CORMEN, T. H. *et al. Algoritmos*: teoria e prática. Rio de Janeiro: Elsevier, 2002.
- DIJKSTRA, E. W. A note on two problems in connexion with graphs. *Numerische Mathematik*, v. 1, p. 269-271, 1959.
- KRUSKAL, J. B. On the shortest spanning subtree of a graph and the traveling salesman problem. *Proceedings of the American Mathematical Society*, v. 7, p. 48-50, 1956.
- SEDGEWICK, R. *Algoritmos em C*. Porto Alegre: Bookman, 2013.
- ZIVIANI, N. *Projeto de algoritmos*: com implementações em Pascal e C. São Paulo: Cengage Learning, 2004.

ATÉ A PRÓXIMA!